Dynamic Programming Series Striver

Topic House Robbery [Lc(213)]

Optimized Approach

```
#include<iostream>
using namespace std;
        int houserobbery(vector<int>&houses,int st,int en){
          int p2=0;
          int p1=houses[st];
        for(int i=st;i<en;i++){
           int keep=houses[i];
           if(i-2>=st)keep+=p2;
           int notkeep=p1;
           int curr=max(keep,notkeep);
           p2=p1;
           p1=curr;
        }
        return p1;
        }
int main(){
vector<int>houses;
int n;
cin>>n;
int j;
vector<int>dp(n);
for(int i=0;i<n;i++){
   cin>>j;
   houses.push_back(j);
}
int maxmprofit=max(houserobbery(houses,0,n-1),houserobbery(houses,1,n));

cout<<maxmprofit<<endl;
   return 0;
}
```

Time Complexity → O(n+n)~=O(2n) ==O(n)

Space Complexity → O(n)